

Algorithmic Knot Theory: Computing the Jones Polynomial

CENG113M Final Project Submission

June 10, 2026

Abstract

This paper outlines the algorithmic computation of the Jones polynomial, a fundamental topological invariant. By discretizing continuous knot diagrams into Planar Diagram (PD) codes, we utilize the Kauffman bracket state-sum model and graph-theoretic smoothings to recursively compute the polynomial invariant using Python.

1 Mathematical Foundations

A mathematical knot is formally defined as an embedding of the 1-sphere into 3-dimensional Euclidean space, $K : S^1 \hookrightarrow \mathbb{R}^3$. Two knots are equivalent if they are related by an ambient isotopy.

Because computation requires discrete logic, we project the 3D knot onto a 2D plane to form a regular projection. By the Reidemeister Theorem, two diagrams represent the same knot if they can be related by a sequence of three planar moves.

1.1 The Kauffman Bracket

The algorithm relies on the Kauffman bracket $\langle D \rangle$, defined axiomatically by a state-sum formula over all possible smoothings of the crossings:

$$\langle D \rangle = \sum_s A^{a(s)-b(s)} (-A^2 - A^{-2})^{|s|-1}$$

where $a(s)$ and $b(s)$ represent the number of A-smoothings and B-smoothings, and $|s|$ represents the number of disjoint unknotted loops.

1.2 The Jones Polynomial

The Kauffman bracket is invariant under Reidemeister Moves II and III, but fails Move I (regular isotopy). To upgrade this to a true ambient isotopy invariant, we calculate the writhe, $w(D)$. The normalized bracket is:

$$X(D) = (-A^3)^{-w(D)} \langle D \rangle$$

Substituting the algebraic variable $A = t^{-1/4}$ yields the universally recognized Jones polynomial, $V_K(t)$.

2 Computational Implementation

The Python implementation maps these abstract algebraic concepts to discrete data structures.

- **Topology to Graph Theory:** We use the `networkx` library to represent the arcs of the knot. Smoothings are applied by adding edges between nodes. The number of disjoint loops $|s|$ is computed by finding the connected components of the graph.
- **Symbolic Algebra:** Standard floating-point arithmetic is insufficient for rigorous mathematics. We deploy a Computer Algebra System (CAS) via the `sympy` library, allowing the program to expand, distribute, and simplify the Laurent polynomials exactly.

The recursive nature of the state-sum evaluates all 2^n possible branchings, successfully computing the formal polynomial.